pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Resampling, SMOTE, and Class Weights

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A rare outcome, and the usual advice

6.97%

bankrupt, training rows

13.4

healthy per bankrupt

Does any fix rank better? What does it do to the output as a number? Where must resampling happen for an honest score?

Four standard fixes

Class weights	same rows, weighted
Random oversampling	copies bankrupt rows
Random undersampling	drops healthy rows
SMOTE	invents bankrupt rows

Tested on Course 4's Polish training rows, by cross-validation inside them.

What "rare" does to a model

0.7543

average precision, pooled out of fold

0.9496

ROC AUC

0.0436

average output; share 0.0697

LightGBM, 200 trees, median imputer, fitted to the rows as they are

Accuracy chooses nothing

Flagging no company is right on 0.9303 of the rows.

Below the share already

Average output 0.0436 against 0.0697: the booster is not calibrated even with no fix.

Keep the average output in mind: every fix today moves it.

How the four fixes work

1. Class weights

every row kept, bankrupt rows count more

2. Oversample

copy bankrupt rows until the classes are equal

3. Undersample

drop healthy rows until they are equal

4. SMOTE

make up new bankrupt rows

You control

Which fix, and how balanced to make the classes (sampling_strategy).

What comes out

A model fitted at a different class balance; its outputs leave the file's scale.

Four ways to make the rare class count

Rows	Healthy	Bankrupt	Total
as they are	4,354	326	4,680
random oversampling	4,354	4,354	8,708
random undersampling	326	326	652
SMOTE	4,354	4,354	8,708

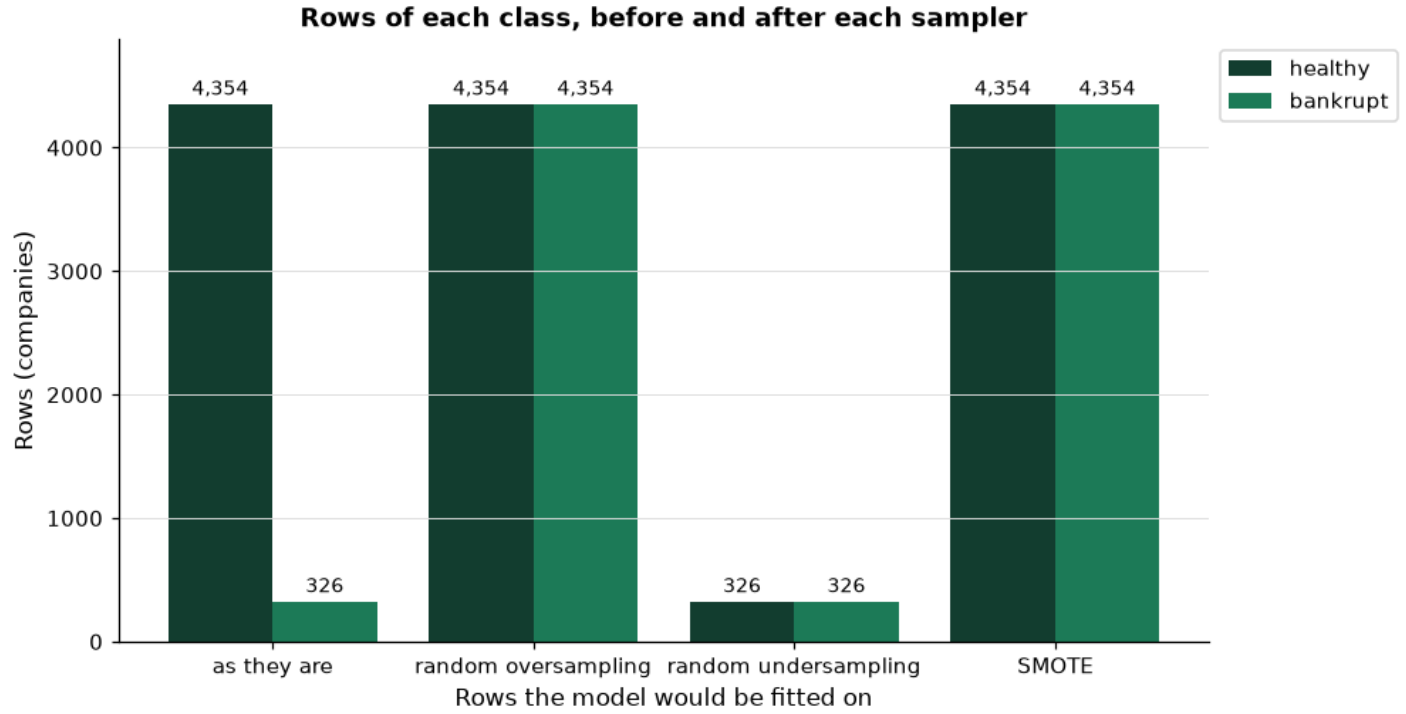
Class weights: no row changes

balanced: a bankrupt row counts 7.1779, a healthy row 0.5374. Both classes carry the same total weight.

```
ensemblekit.class_counts_after(ImbPipeline([("impute", fill()), ("resample", SMOTE(random_state=SEED))]), X_train, y_train)
```

A demonstration: counting what a sampler does. No model is scored on these rows.

Rows of each class, before and after



Add rows or drop them

Oversampling and SMOTE: 4,354 healthy, 4,354 bankrupt. Undersampling: 326 and 326.

In Excel: a clustered column chart of the same table.

A demonstration: no model is fitted or scored on these rows.

How SMOTE works

1. Pick a company

a bankrupt row

2. Find neighbours

the five nearest bankrupt rows

3. Draw a point

x + u * (neighbour - x), u between 0 and 1

4. Repeat

until the classes are equal

You control

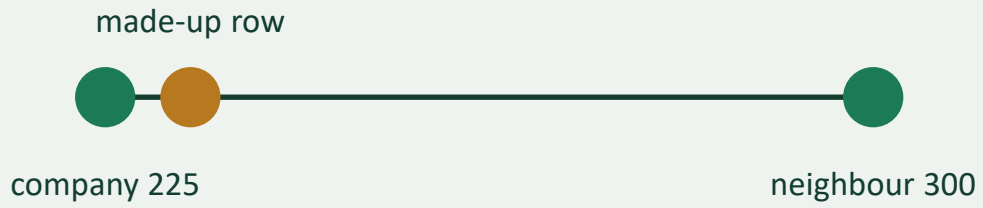
k_neighbors, sampling_strategy, and the seed.

What comes out

More fitting rows; every new bankrupt row is invented.

SMOTE's synthetic row: a point between two companies

u = 0.110998



Found from imbalanced-learn's output: 4,028 made-up rows, 8,708 rows in all. All 64 columns of the first one within 1e-12.

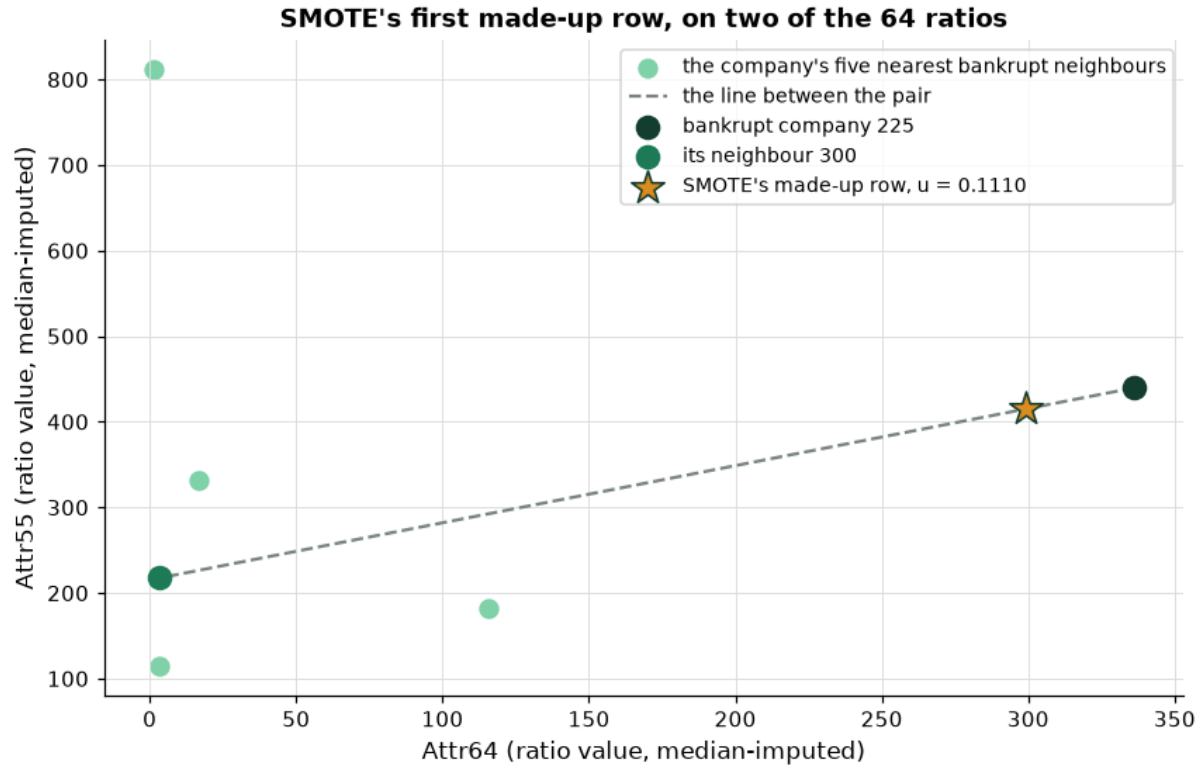
How SMOTE makes a row

Take a bankrupt company, pick one of its five nearest bankrupt neighbours, draw u between 0 and 1. Every column moves the same fraction u.

```
x + u * (neighbour - x)
```

A made-up row is a point between two companies that existed, not a company.

The made-up row on two of the 64 ratios



Attr64 and Attr55

The two ratios where company 225 and neighbour 300 differ most. The star sits on the dashed line, 11.1 percent of the way from the company, as in all 64 columns.

No real company sits at the star.

Where the name comes from

N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, June 2002

"SMOTE: Synthetic Minority Over-sampling Technique"

Journal of Artificial Intelligence Research, volume 16, pages 321 to 357

The title says what section 13.3 found: the rare class oversampled with rows made up, not copied.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1613/jair.953). Bibliographic facts only; the record gives no issue number; the paper itself was not read for this course.

How resampling stays inside the folds

1. Cut folds first

before any resampling

2. Resample training

only while fitting

3. Score held-out

as it is, every company real

4. No leak

no copy or neighbour of a held-out row

You control

The steps and their order in the ImbPipeline.

What comes out

A cross-validated score with no resampled row in it.

Resampling inside the pipeline, and the trap

SMOTE before cross-validation

0.9979

8,708 rows scored, 4,028 made up, half bankrupt. ROC AUC 0.9982.

SMOTE as a step of the ImbPipeline

0.6471

4,680 real rows scored, each once. ROC AUC 0.9272.

Pooled out-of-fold average precision, the same LightGBM, the same SMOTE, the same seed

Fold 1: 3,744 training rows (261 bankrupt) become 6,966 after SMOTE; 936, held-out rows scored as they are.

A sampler is a step of an ImbPipeline, after the imputer, before the model. Nothing is resampled before the folds.

How the models are ranked

1. Same five folds

for every model

2. Pool the outputs

one out-of-fold probability per row

3. One AP

over all the pooled rows

4. Rank the eight

highest first

You control

The folds, the models, and the score.

What comes out

One pooled score per model, and a ranking of the eight.

Ranking: eight models, one set of folds

Model	Average precision	ROC AUC
LightGBM	0.7543	0.9496
LightGBM, balanced weights	0.7483	0.9437
LightGBM, oversampled	0.7595	0.9492
LightGBM, undersampled	0.5035	0.9240
LightGBM, SMOTE	0.6471	0.9272
Logistic	0.6841	0.9202
Logistic, balanced (Course 4)	0.6497	0.9264
Logistic, SMOTE	0.6597	0.9212

On this file, these models

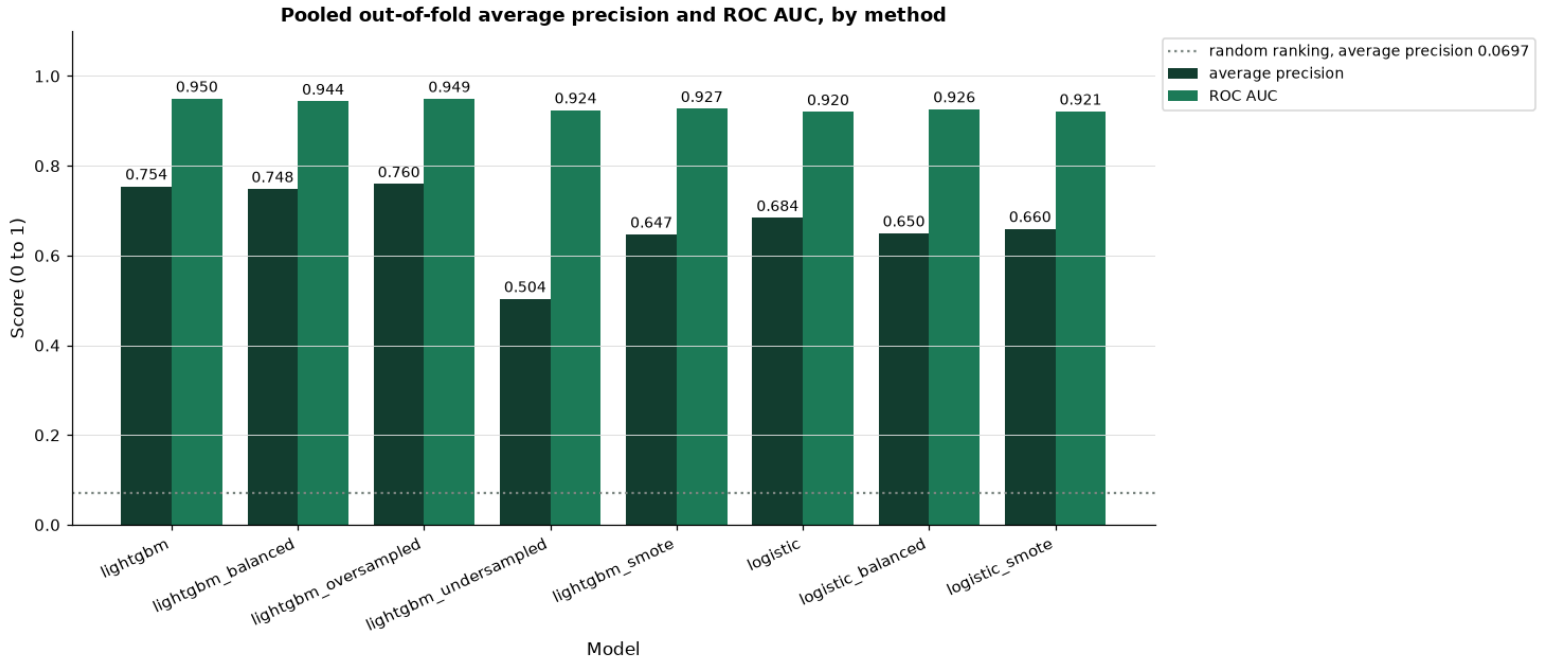
No fix ranks better than the rows as they are. SMOTE and undersampling cost the most.

Pooled against fold mean

Course 4's model: fold mean 0.6624 (Course 4's number), pooled 0.6497.

A finding about these rows, not a law: weights and resampling move outputs more than they reorder them.

The ranking, drawn



Which bar separates

0.5035

undersampled

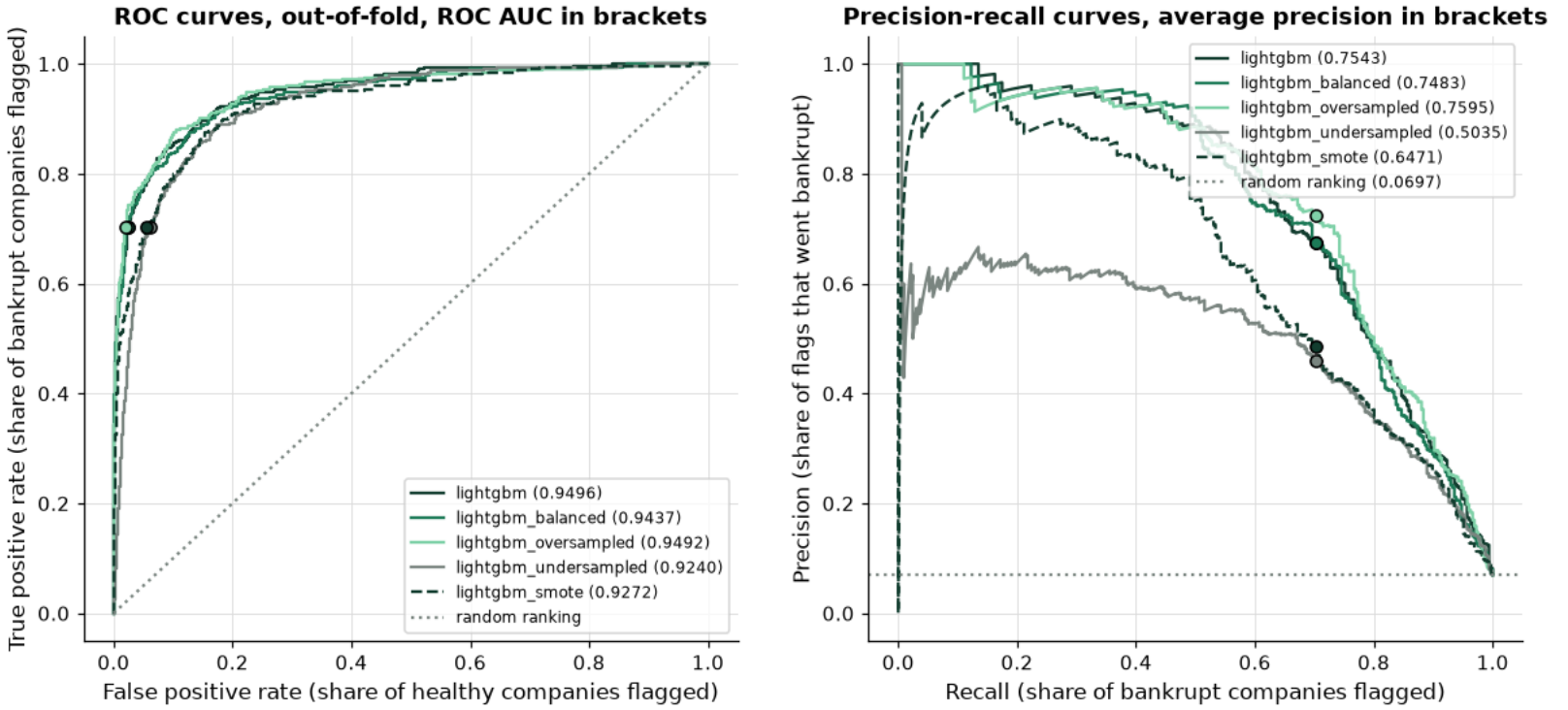
0.6471

SMOTE

Average precision, LightGBM: the two shortest bars. Every ROC AUC bar is above 0.92.

On a rare outcome, read average precision first.

ROC and precision-recall curves, five LightGBM models



The dots: recall 0.7025 for every model, precision from 0.4589 (undersampled) to 0.7247 (random oversampling).

How the Brier score works

1. Average output

should be near the bankrupt share

2. Squared gap

output minus outcome, 0 or 1

3. Average the gaps

lower is better

4. Order, not value

ranking uses order, Brier uses values

You control

Only the outputs and the outcomes.

What comes out

One Brier score per model, read beside its average output.

What each fix does to the output's average

Model	Average output	Brier score
LightGBM	0.0436	0.0344
LightGBM, balanced weights	0.0520	0.0342
LightGBM, oversampled	0.0519	0.0324
LightGBM, undersampled	0.2081	0.1261
LightGBM, SMOTE	0.0747	0.0433
Logistic	0.0707	0.0350
Logistic, balanced (Course 4)	0.2091	0.0850
Logistic, SMOTE	0.1947	0.0814

0.0697

bankrupt share

Allowed names

probability

model's score

flag at t

Never called

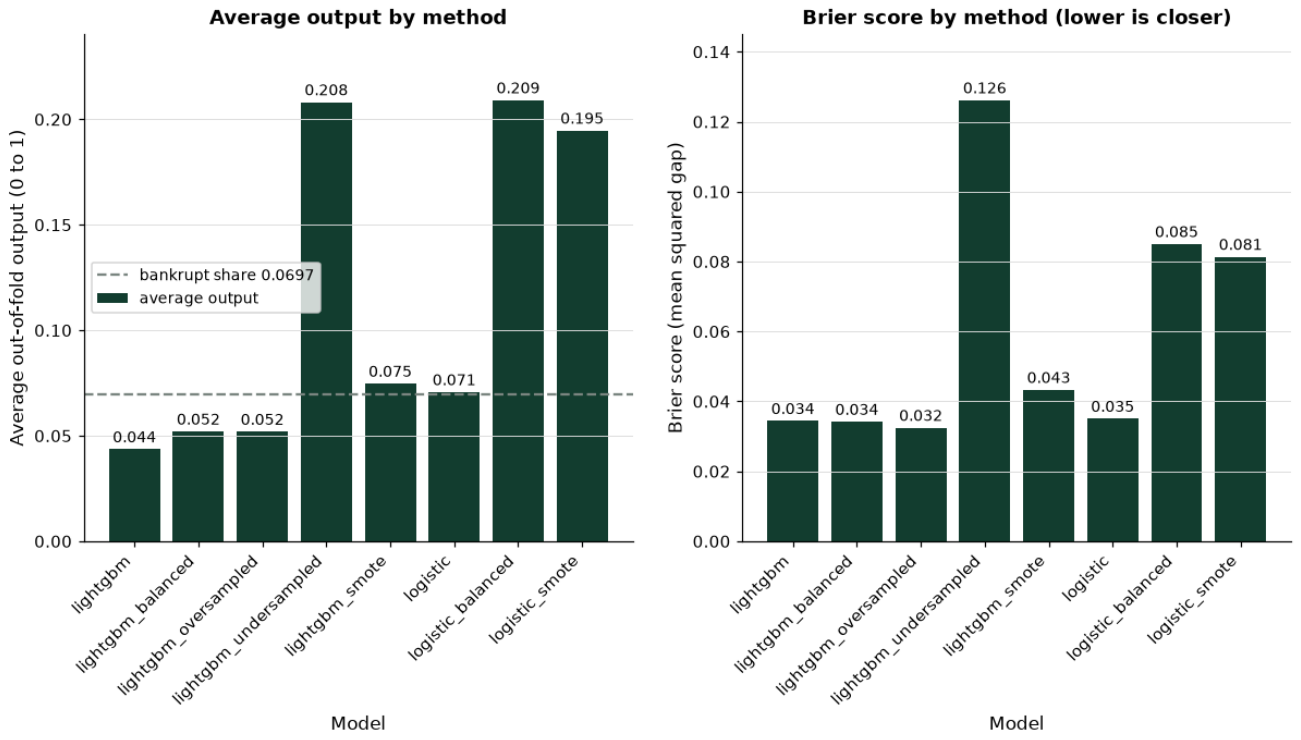
credit view

rating

decision

Weights and resampling fit the model to rows that look half bankrupt: its output is the model's score.

The average and the Brier score, drawn



The three tallest bars on the left carry the three tallest Brier bars: level, not ranking skill.

How a threshold makes models comparable

1. Fix the recall

at least 0.70

2. Find threshold

highest one that reaches it, out of fold

3. Count the flags

companies flagged at that threshold

4. Compare the cost

flagged, and the share that went bankrupt

You control

The recall target and the folds.

What comes out

One threshold per model, with its flagged count and precision.

The same recall from a threshold

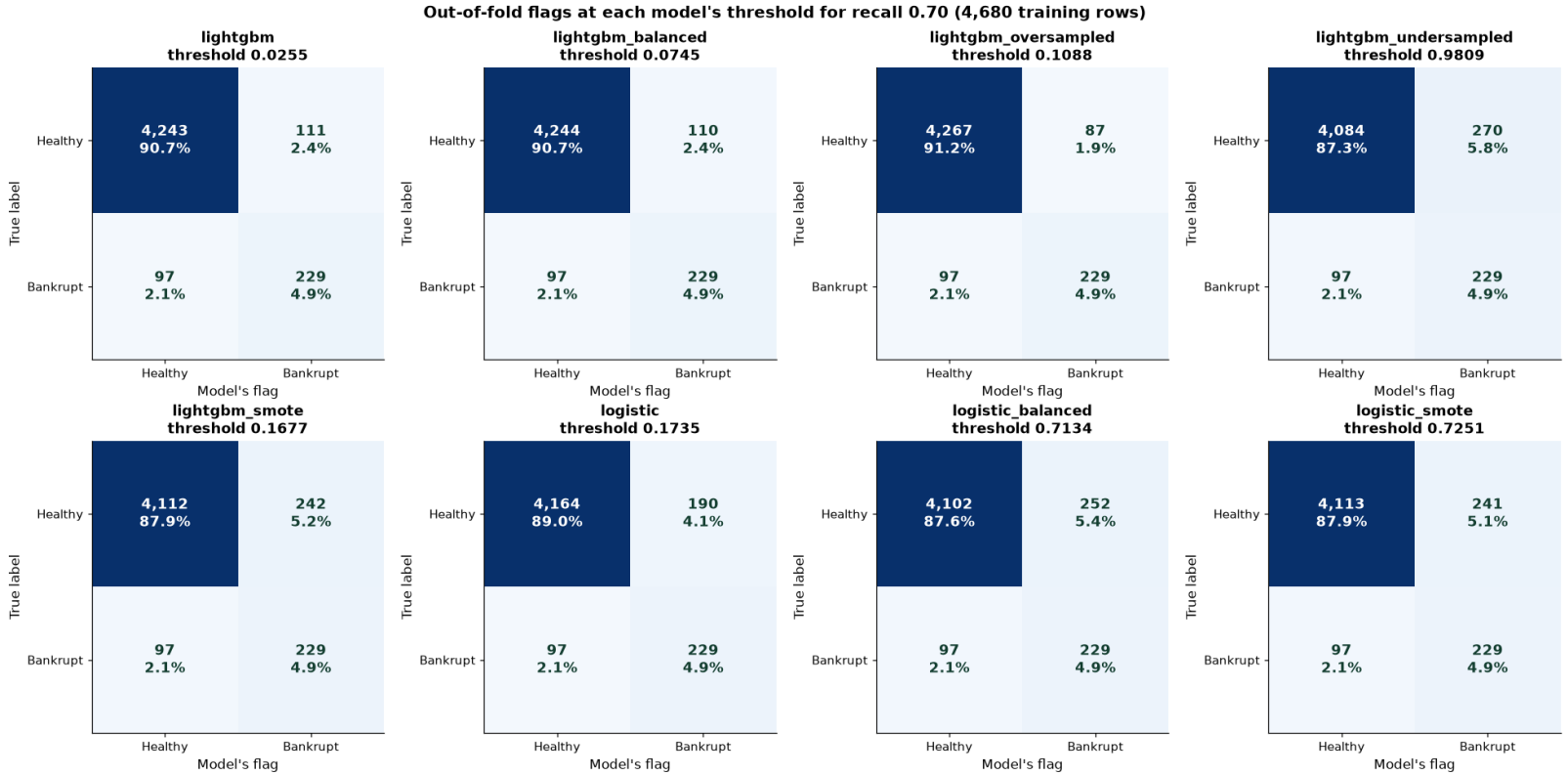
Model	Threshold	Flagged	Precision
LightGBM	0.0255	340	0.6735
LightGBM, balanced weights	0.0745	339	0.6755
LightGBM, oversampled	0.1088	316	0.7247
LightGBM, undersampled	0.9809	499	0.4589
LightGBM, SMOTE	0.1677	471	0.4862
Logistic	0.1735	419	0.5465
Logistic, balanced (Course 4)	0.7134	481	0.4761
Logistic, SMOTE	0.7251	470	0.4872

Every model: recall 0.7025

The threshold absorbs the change in level. SMOTE costs 131 extra flags for the same bankrupt companies.

Move the threshold, not the data: the unweighted output stays a probability.

Eight confusion heatmaps, one rule



The cost of recall

Bottom right: 229 caught in every panel. Top right, healthy companies flagged: 87 to 270.

The top-right cell is the cost of the recall.

How recalibration works

1. Prior correction

multiply the odds by one fixed number

2. Order unchanged

every ranking metric stays the same

3. Isotonic

learn a step function from score to observed share

4. Check

calibration curve and Brier score

You control

The method and, for isotonic, the folds.

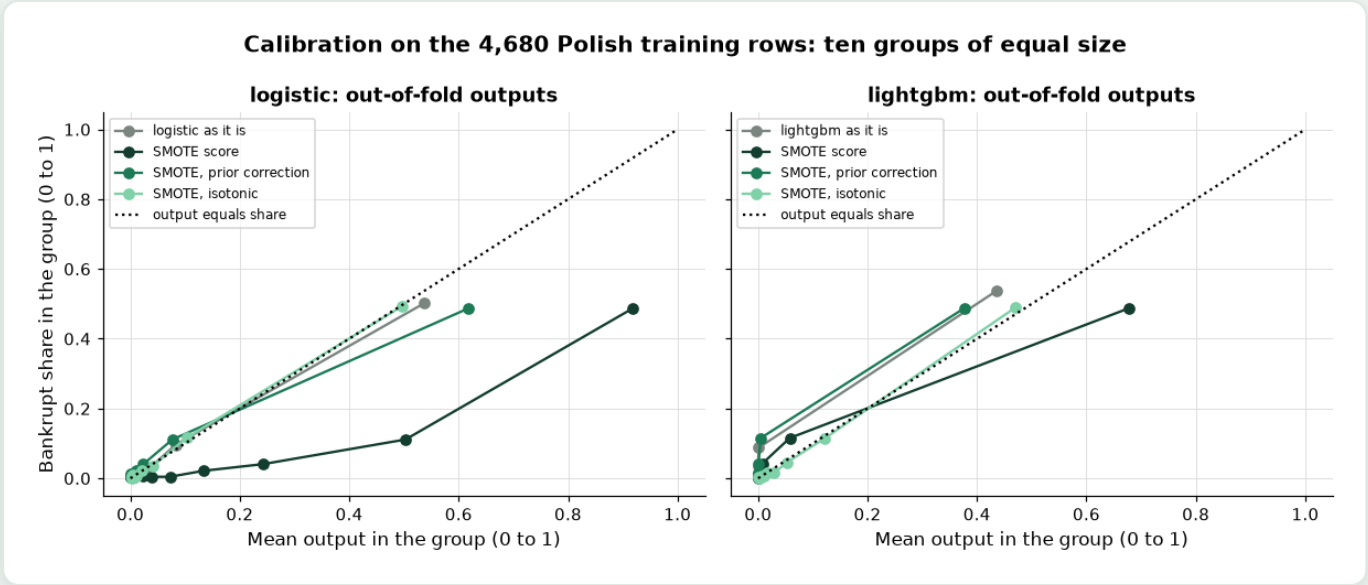
What comes out

Outputs back on the file's scale.

Recalibrating a resampled score

Logistic, SMOTE	Average	Brier
Score	0.1947	0.0814
Prior correction	0.0742	0.0390
Isotonic	0.0700	0.0361
Logistic as it is	0.0707	0.0350

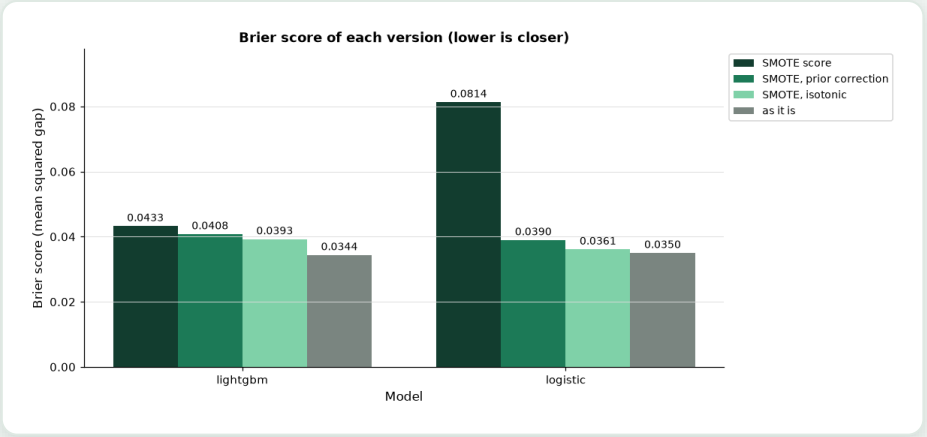
LightGBM, SMOTE: prior correction 0.0382 (below the share), isotonic 0.0712; Brier 0.0408 and 0.0393. Isotonic average precision 0.6793 and 0.6217: a new model.



Among the logistic regressions, the lowest Brier score: the one never resampled or weighted.

Three versions of each SMOTE score

Model	Version	Average	Brier	Avg precision
Logistic	SMOTE score	0.1947	0.0814	0.6597
Logistic	SMOTE, prior correction	0.0742	0.0390	0.6597
Logistic	SMOTE, isotonic	0.0700	0.0361	0.6793
Logistic	as it is	0.0707	0.0350	0.6841
LightGBM	SMOTE score	0.0747	0.0433	0.6471
LightGBM	SMOTE, prior correction	0.0382	0.0408	0.6471
LightGBM	SMOTE, isotonic	0.0712	0.0393	0.6217
LightGBM	as it is	0.0436	0.0344	0.7543



Recalibration repairs part of what SMOTE did; the model as it is stays lower.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIF(A2:A4681,0) and 1`	before: 4,354 and 326
`=COUNTIF(B2:B8709,0) and 1`	after SMOTE: 4,354 and 4,354
`=B2+$A$6*(B3-B2), filled across`	SMOTE's made-up row, 64 columns
`=(E1/(1-E1))/(E2/(1-E2))`	k = 0.074874
`=B2*$E$3/(B2*$E$3+1-B2), filled down`	prior_correction, row for row
`=AVERAGE(B2:B4681)`	average 0.1947 -> 0.0742
`=SUMXMY2(B2:B4681,A2:A4681)/ROWS(A2:A4681)`	Brier 0.0814 -> 0.0390

No SMOTE, no class-weighted fit, no cross-validation in Excel: the arithmetic around them, every value within 1e-12.

Save the day: git log --decorate

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
lightgbm,cv,0.9496,0.7543,0.0255,0.9556,0.6735,0.7025,0.6877
lightgbm_balanced,cv,0.9437,0.7483,0.0745,0.9558,0.6755,0.7025,0.6887
lightgbm_oversampled,cv,0.9492,0.7595,0.1088,0.9607,0.7247,0.7025,0.7134
lightgbm_undersampled,cv,0.9240,0.5035,0.9809,0.9216,0.4589,0.7025,0.5552
lightgbm_smote,cv,0.9272,0.6471,0.1677,0.9276,0.4862,0.7025,0.5747
logistic,cv,0.9202,0.6841,0.1735,0.9387,0.5465,0.7025,0.6148
logistic_balanced,cv,0.9264,0.6497,0.7134,0.9254,0.4761,0.7025,0.5675
logistic_smote,cv,0.9212,0.6597,0.7251,0.9278,0.4872,0.7025,0.5754
```

```
git -C "{work}" log --oneline --decorate
```

(HEAD -> main): HEAD is where you are, main the branch it follows.

Every threshold out of fold, every recall the same; accuracy beside the baseline 0.9303. 50 tests pass.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: SMOTE runs with fit_resample before the folds are cut, so made-up rows are scored.

Code that runs is not the same as code that is right.

What you can do now

- Four ways to make a rare class count
- A SMOTE row: a point between two companies
- Samplers go inside an ImbPipeline
- The same recall from a threshold, fewer flags
- A resampled output is a score until recalibrated

Week 3, Day 4 SHAP values, and what they do not say